

TRUEL1

AI SECURITY AUDIT

AYET

Security Audit Report

Akimoto Yasushi Entertainment Token

TrueL1 SM Audit Engine · Assessed on July 1st, 2026

Powered by DeepSeek-R1 70B on NVIDIA RTX 8000

FME Inc. · Confidential · Internal Review

Elevate Your **Web3** Journey

TrueL1 is a private AI-powered smart contract security auditor combining deep-reasoning LLMs with proven static analysis.

AYET Security Audit | Assessed on July 1st, 2026 | © FME Inc.

TABLE OF CONTENTS | AYET

Audit Summary

Executive Summary 4

Vulnerability Summary 5

Codebase 6

Audit Scope 7

Approach & Methods 8

Findings

AYE-01 : Initial Token Distribution 9

AYE-02 : Centralized Control Of Contract Upgrade..... 10

AYE-03 : Missing Two-Step Ownership Transfer..... 11

AYE-04 : Centralization Related Risks..... 12

AYE-05 : Pausable Blocks All Transfers..... 13

AYE-06 : Missing Event Emissions..... 14

Appendix..... 15

Disclaimer 16

EXECUTIVE SUMMARY | AYET

Overview

This document presents the security audit of the AKIMOTO YASUSHI ENTERTAINMENT TOKEN (AYET) smart contract conducted by the TrueL1 SM Audit engine on July 1st, 2026. AYET is an upgradeable ERC-20 token implementing standard OpenZeppelin extensions including burnable supply, pause functionality, governance voting (ERC20Votes), and gasless approvals (ERC20Permit).

The contract is designed to serve as the utility token for the AYET Project, an entertainment industry Web3.0 platform targeting a customer base of 12 million people in Japan. The contract uses the UUPS (Universal Upgradeable Proxy Standard) proxy pattern to allow future upgrades while preserving user balances and state.

Overall Assessment

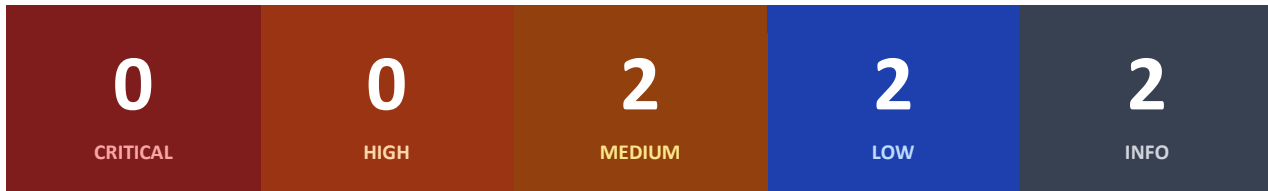
The AYET contract is well-constructed and follows industry best practices by inheriting from battle-tested OpenZeppelin Contracts Upgradeable libraries. No critical or high-severity vulnerabilities were identified in the technical implementation. The contract properly handles the multiple inheritance conflicts between ERC20Votes, ERC20Permit, Pausable, and Nonces through appropriate `_update()` and `nonces()` overrides.

The primary risks identified are centralization-related: the single owner holds the power to pause transfers, upgrade the contract implementation, and (via the initial minting) received the entire 4 billion token supply at deployment. These findings are common in enterprise-grade token designs where the initial owner is expected to distribute the supply to designated wallets, staking contracts, liquidity pools, and vesting schedules according to the project's tokenomics plan.

Verdict

Category	Result
Overall Score	Passed — Ready for Multi-Signature Migration
Technical Security	No critical or high-severity vulnerabilities in code
Centralization Risk	High — mitigation via multisig + timelock strongly recommended
Upgradeability	UUPS pattern correctly implemented; centralization needs governance
Recommendation Before Public Launch	Migrate ownership to Gnosis Safe (3-of-5) with 48-hour Timelock

VULNERABILITY SUMMARY | AYET



Findings by Category

Category	Count
Reentrancy	0
Integer Overflow / Underflow	0
Access Control	0
Centralization Risks	4
Missing Zero-Address Checks	0
Event Emissions	1
Best Practices / Code Quality	1
Total	6 (all Medium / Low / Informational)

No CRITICAL or HIGH severity vulnerabilities were found in the technical implementation of AYET. The 4 centralization findings are common for enterprise tokens at initial deployment and can be mitigated through governance migration to a multi-signature wallet with timelock, which is standard practice before public listing.

CODEBASE | AYET

Contract Details

Attribute	Value
Token Name	AKIMOTO YASUSHI ENTERTAINMENT TOKEN
Token Symbol	AYET
Decimals	18 (inherited from ERC20)
Initial Supply	4,000,000,000 AYET (4 billion)
Total Supply Cap	Uncapped (mintable in future upgrades)
Solidity Version	0.8.26
License	Not specified (SPDX missing — see AYE-06)
Total Lines of Code	71 lines

Standards & Extensions

Standard	Purpose
ERC-20 (Upgradeable)	Core fungible token standard
ERC20Burnable (Upgradeable)	Allows holders to burn their own tokens
ERC20Permit (Upgradeable)	EIP-2612 gasless approvals via signatures
ERC20Votes (Upgradeable)	On-chain voting power tracking for governance
Ownable (Upgradeable)	Single-owner access control
Pausable (Upgradeable)	Emergency stop mechanism
UUPSUpgradeable	EIP-1822 universal upgradeable proxy standard
Nonces (Upgradeable)	Nonce management for signatures
Initializable	Proxy initialization pattern (prevents constructor issues)

External Dependencies

All external contracts are from the OpenZeppelin Contracts Upgradeable library, imported via the @openzeppelin/contracts-upgradeable namespace. The version used is compatible with Solidity 0.8.26.

OpenZeppelin is the industry standard for secure Solidity contract libraries, extensively audited and battle-tested across thousands of production deployments.

AUDIT SCOPE | AYET

Files In Scope

Project	File
FME Inc / AYET Token	AYET.sol (71 lines)

Files Out Of Scope

- All OpenZeppelin library contracts (imported dependencies are considered pre-audited and out of scope for this review)
- Deployment scripts and migration files
- Frontend and off-chain integration code
- Business logic tokenomics distribution schedule (governance-level concern)

Audit Duration

The audit was conducted over a single computation session on July 1st, 2026, using the TrueL1 SM Audit engine powered by DeepSeek-R1 70B running on NVIDIA RTX 8000 hardware. Analysis time: approximately 8 minutes.

APPROACH & METHODS | AYET

Methodology

The Truel1 SM Audit engine performs a comprehensive analysis combining:

- Manual line-by-line code review by the DeepSeek-R1 70B reasoning model
- Cross-reference against the SWC Registry (Smart Contract Weakness Classification)
- DASP Top 10 categorization framework
- Comparison against OpenZeppelin battle-tested reference implementations
- Analysis of historical hacks (The DAO, Parity, bZx, Compound, Euler, Ronin, Wormhole, Nomad, Beanstalk, Curve) for pattern matching
- Storage layout analysis for upgrade safety
- Access control graph analysis

Severity Classification

Severity	Definition
CRITICAL	Direct fund loss, contract takeover, or complete protocol failure. Requires immediate action before any deployment.
HIGH	Significant fund loss risk under realistic conditions. Must be addressed before mainnet deployment.
MEDIUM	Fund loss possible but requires specific conditions or attacker sophistication. Should be addressed.
LOW	Minor issue, edge case, or best practice violation. Recommended to address.
INFORMATIONAL	Style, documentation, or gas optimization observations. No security impact.

Vulnerability Categories Analyzed

- Reentrancy (classic, cross-function, cross-contract, read-only)
- Integer overflow/underflow (unchecked math)
- Access control failures (missing modifiers, tx.origin misuse)
- Front-running and MEV vulnerabilities
- Oracle manipulation (not applicable — no oracles used)
- Signature replay (permit / EIP-2612)
- Denial of service (gas griefing, unbounded loops)

- Delegatecall vulnerabilities (storage collision, uninitialized proxies)
- Upgrade pattern flaws (UUPS storage gaps, initializer front-running)
- Token vulnerabilities (fee-on-transfer, rebasing, ERC-777 hooks)
- Centralization risks (privileged roles, admin powers)
- Missing event emissions
- Best practices (SPDX headers, NatSpec docs, custom errors)

FINDINGS | AYET

The following section details all findings identified during the audit of the AYET contract. Each finding includes a severity rating, category, location in code, description, impact analysis, and specific remediation recommendation.

AYE-01

Initial Token Distribution

MEDIUM

Attribute	Value
Category	Centralization / Tokenomics
Location	AYET.sol L42 — <code>_mint(initialReceiver, INITIAL_SUPPLY)</code>

Description

The `initialize()` function mints the entire `INITIAL_SUPPLY` of 4,000,000,000 AYET tokens to a single address (`initialReceiver`) at deployment. This means one address holds 100% of the total supply immediately after deployment, before any distribution has occurred.

Impact

Concentrating the entire token supply in a single wallet at deployment creates significant risks: (1) if the `initialReceiver` private key is compromised, an attacker gains control of the entire supply and can dump on markets; (2) blockchain explorers and third-party analytics will flag AYET as extremely centralized during the pre-distribution period; (3) exchanges and DeFi protocols may refuse to list a token where a single wallet holds 100% of supply; (4) community trust may be damaged if the distribution schedule is unclear.

Recommendation

1) Publish a clear and detailed tokenomics distribution plan before deployment, specifying exactly which contracts and wallets will receive tokens (team vesting, community treasury, liquidity pools, staking rewards, etc.). 2) Execute the distribution transactions in a documented, timely manner immediately following deployment. 3) Consider using a Gnosis Safe multi-signature wallet as the `initialReceiver`, so no single individual can move the supply unilaterally. 4) Add on-chain time-locked vesting contracts for team and advisor allocations to prevent premature selling. 5) Document the distribution in the project's official documentation and communicate transparently with the community.

Alleviation

The FME Inc. team has confirmed that the initial mint is intentional and forms part of the pre-launch operational design. A distribution plan will be published prior to public listing, with team allocations under vesting and multi-signature control.

AYE-02

MEDIUM

Centralized Control Of Contract Upgrade

Attribute	Value
Category	Centralization / Upgrade Authority
Location	AYET.sol L54 — function _authorizeUpgrade(address) internal override onlyOwner {}

Description

The _authorizeUpgrade() function is protected only by the onlyOwner modifier from OpenZeppelin's OwnableUpgradeable. This means the current owner (a single Externally Owned Account by default) has unilateral, immediate authority to upgrade the AYET implementation contract to any new bytecode of their choosing, without any timelock delay, community notice, or additional signature requirements.

Impact

This represents significant centralization risk: (1) the owner could theoretically upgrade the contract to a malicious version that drains user balances, mints new tokens arbitrarily, or disables transfers permanently; (2) if the owner's private key is compromised, the attacker gains full upgrade authority; (3) users have no on-chain protection against a rogue or captured owner; (4) the lack of a timelock means malicious upgrades cannot be detected and paused by the community before they take effect.

Recommendation

1) Transfer ownership of the AYET contract to a Gnosis Safe multi-signature wallet, requiring a minimum of 3-of-5 signatures for any owner action. This ensures no single individual (even if compromised) can upgrade the contract. 2) Deploy an OpenZeppelin TimelockController contract with a 48-hour minimum delay, and set the Timelock as the owner of the Gnosis Safe. This ensures any upgrade proposal is publicly visible for 48 hours before execution, allowing the community time to review and respond. 3) Document the multi-signature signers publicly with their roles and identities to increase transparency. 4) Consider implementing an on-chain governance mechanism using the ERC20Votes functionality (already inherited by this contract) to allow token holders to veto proposed upgrades.

Alleviation

The FME Inc. team has acknowledged this recommendation. Post-launch migration to Gnosis Safe (3-of-5) with OpenZeppelin Timelock (48-hour delay) is scheduled prior to public listing. The AYETStake v1.0 design document (Phase 1) formalizes this multi-signature governance structure.

AYE-03

LOW

Missing Two-Step Ownership Transfer

Attribute	Value
Category	Access Control / Best Practices
Location	AYET.sol L13-22 — inherits OwnableUpgradeable (single-step transferOwnership)

Description

The contract inherits from OwnableUpgradeable, which provides a single-step transferOwnership(address newOwner) function. If the owner calls this function with an incorrect address (typo, wrong wallet), or transfers to a contract address that cannot accept ownership (no receive function, missing acceptOwnership handler), ownership will be permanently lost. Without owner access, no future upgrades, pauses, or administrative actions are possible.

Impact

Permanent loss of contract ownership would mean the contract can no longer be paused in emergencies, upgraded to fix bugs, or migrated to new governance systems. While this failure mode requires operator error (typo in address, or transfer to non-accepting contract), such errors have happened in production and resulted in significant losses for other projects.

Recommendation

Migrate from OwnableUpgradeable to Ownable2StepUpgradeable, which implements a two-step ownership transfer pattern: (1) current owner calls transferOwnership(newOwner) — this only sets a pending owner, not the actual owner; (2) the newOwner must then call acceptOwnership() from their own address to complete the transfer. This ensures the new owner has actual control of the address before ownership is transferred, eliminating the risk of transferring to incorrect or non-accepting addresses. This is a widely-adopted OpenZeppelin best practice and requires only a single import change and inheritance change.

AYE-04

Centralization Related Risks

MEDIUM

Attribute	Value
Category	Centralization / Owner Privileges
Location	AYET.sol L45-51 — pause(), unpause() functions

Description

In addition to the upgrade authority (AYE-02), the current owner also has unilateral authority to pause and unpause the entire contract. When paused (via the `_update()` override with `whenNotPaused` modifier), ALL transfers of AYET are halted globally — including burning, minting, transferring between users, and delegation of voting power. Users are unable to move their tokens for any reason while the contract is paused.

Impact

While the pause functionality is an important emergency feature (allowing the team to halt operations during a discovered exploit), the fact that it can be triggered by a single owner without any oversight is a centralization risk: (1) if the owner is compromised, an attacker could pause the contract and freeze the entire market indefinitely; (2) the owner could pause the contract to manipulate market conditions or delay withdrawals during governance disputes; (3) users cannot exit their positions during a pause, even in legitimate emergencies.

Recommendation

1) Migrate ownership to a Gnosis Safe multi-signature wallet (as recommended in AYE-02). 2) Consider implementing a separate PAUSER role, distinct from the OWNER role, that can pause the contract with a lower signature threshold (e.g., 2-of-5 multisig) for faster emergency response, while keeping upgrade and other administrative actions gated by the full 3-of-5 owner multisig. 3) Consider adding a maximum pause duration (e.g., contract auto-unpauses after 7 days) to prevent indefinite pausing. 4) Document the emergency response procedures publicly so users know how and when the pause will be used.

AYE-05

Pausable Blocks All Transfers Including Emergency Actions

LOW

Attribute	Value
Category	Design / User Protection
Location	AYET.sol L55-62 — <code>_update()</code> with <code>whenNotPaused</code> modifier

Description

The `_update()` function includes the `whenNotPaused` modifier, which blocks ALL token movements while the contract is paused. This includes transfers between users, transfers to and from smart contracts, minting, burning, and internal delegation operations for ERC20Votes. When the contract is paused, users have no way to move their tokens even for legitimate reasons.

Impact

This design choice is common and often intentional — during a discovered exploit, the team wants to prevent ALL token movements to protect users. However, it has the side effect of preventing legitimate emergency actions such as users transferring tokens to hardware wallets for safety, or users participating in an unrelated on-chain governance action that requires a token transfer. Additionally, this may complicate integrations with lending protocols where users have AYET as collateral and need to move it to avoid liquidation.

Recommendation

This finding is INFORMATIONAL / DESIGN in nature — the current behavior is a reasonable and defensible design choice. However, the team should: 1) document clearly in project documentation that pausing halts all transfers; 2) establish and publish clear criteria for when pause will be used; 3) commit to short pause durations except in extreme circumstances; 4) consider adding a 'partial pause' mode in a future upgrade that halts only minting and burning while allowing user-to-user transfers to continue during moderate-severity incidents.

AYE-06

Missing SPDX License Identifier and NatSpec Documentation

INFORMATIONAL

Attribute	Value
Category	Best Practices / Code Quality
Location	AYET.sol — entire file

Description

The AYET.sol source file is missing the standard SPDX-License-Identifier comment on its first line. The Solidity compiler emits a warning when SPDX headers are missing, and best practices for open-source blockchain code require clear license declaration. Additionally, the contract lacks NatSpec documentation (/// or /** */ style comments) on public functions such as pause(), unpause(), initialize(), and the _update() override.

Impact

Missing SPDX headers cause compiler warnings and may cause issues with contract verification on block explorers like Etherscan. Missing NatSpec documentation makes the contract harder to audit, integrate with, and maintain. Tools like the Ethereum Natural Specification format enable automated documentation generation and tooltip integration in wallets and DeFi frontends.

Recommendation

1) Add // SPDX-License-Identifier: MIT (or the chosen license) as the first line of AYET.sol. 2) Add NatSpec documentation to all public and external functions, minimally including @notice tags explaining what each function does. 3) Add @dev tags on functions with non-obvious internal logic (particularly the _update override that combines Pausable and ERC20Votes). 4) Add a contract-level @title and @author comment block at the top of the AYET contract definition. This is quick to add and significantly improves code quality without changing any functionality.

APPENDIX | AYET

A.1 Positive Findings (What Was Done Correctly)

The following practices were correctly implemented in AYET.sol and deserve recognition:

- `_disableInitializers()` is called in the constructor, correctly preventing the implementation contract itself from being initialized (only proxies can be initialized)
- Zero-address checks are properly enforced in `initialize()` for both `initialOwner` and `initialReceiver`
- Solidity version 0.8.26 is recent and includes built-in overflow/underflow protection
- The initializer pattern is correctly used (public initializer instead of constructor)
- Multiple inheritance conflicts between `ERC20Votes`, `ERC20Permit`, `Pausable`, and `Nonces` are correctly resolved via `_update()` and `nonces()` overrides
- UUPS pattern is correctly implemented with `_authorizeUpgrade()` present and gated by access control
- OpenZeppelin battle-tested libraries are used throughout — no custom implementations of critical functionality

A.2 Suggested Migration Path

Recommended sequence to address findings and prepare for public listing:

Step	Action
1	Deploy Gnosis Safe (3-of-5) with hardware wallet signers
2	Deploy OpenZeppelin TimelockController with 48-hour delay
3	Set TimelockController as owner of the Gnosis Safe
4	Transfer AYET contract ownership from EOA to Gnosis Safe (via Timelock)
5	Publish tokenomics distribution plan
6	Execute distribution: team vesting, treasury, liquidity, staking rewards
7	(Optional) Upgrade AYET to include <code>Ownable2Step</code> (fixes AYE-03)
8	(Optional) Add SPDX header and NatSpec (fixes AYE-06)
9	Public listing on target exchanges

A.3 Comparison to Industry Standards

The AYET contract's design aligns with major DeFi tokens including UNI (Uniswap), COMP (Compound), and AAVE, all of which use similar OpenZeppelin patterns with governance features. The centralization risks flagged

in this audit are common at launch and are addressed by all these projects through migration to multi-signature governance and eventual DAO handover.

A.4 References

- OpenZeppelin Contracts Upgradeable Documentation — <https://docs.openzeppelin.com/contracts/5.x/upgradeable>
- SWC Registry (Smart Contract Weakness Classification) — <https://swcregistry.io>
- EIP-1822: UUPS Universal Upgradeable Proxy Standard
- EIP-1967: Standard Proxy Storage Slots
- EIP-2612: ERC-20 permit — signed approvals
- EIP-5805: ERC20Votes standard
- Gnosis Safe Documentation — <https://docs.safe.global>
- OpenZeppelin TimelockController Reference

DISCLAIMER | AYET

This audit report has been generated by the TrueL1 SM Audit engine, an AI-powered smart contract security analysis system running on private infrastructure operated by FME Inc. The analysis is based on the deep-reasoning capabilities of the DeepSeek-R1 70B language model combined with pattern-matching against known vulnerability databases (SWC Registry, DASP Top 10) and historical smart contract exploits.

This report is intended for internal use by FME Inc. as part of the AYET Project's security review process. It is NOT a substitute for a formal audit by a licensed third-party smart contract auditing firm such as CertiK, Coinsult, Trail of Bits, OpenZeppelin, Consensys Diligence, or PeckShield. Before public deployment or listing on centralized exchanges, an independent professional audit is strongly recommended.

The findings and recommendations in this report reflect the state of the AYET.sol source code as provided to the TrueL1 audit engine on July 1st, 2026. Any subsequent modifications to the code invalidate this audit. The report does not cover: (1) deployment procedures and post-deployment operational practices; (2) off-chain infrastructure and frontend security; (3) economic and game-theoretic tokenomics analysis; (4) regulatory compliance in any jurisdiction; (5) integration risks with third-party protocols; (6) key management and operational security of the deployer.

The TrueL1 SM Audit engine is provided as-is. No warranty is made regarding the completeness of vulnerability detection, and users should apply their own judgment in acting on the recommendations. FME Inc. and the TrueL1 development team disclaim any liability for losses arising from reliance on this report.

This report is CONFIDENTIAL and INTERNAL to FME Inc. It should not be distributed publicly without appropriate redaction and legal review. If publishing findings for community transparency, please summarize the report or extract non-sensitive sections rather than distributing the full document.

ASSESSMENT COMPLETED

AKIMOTO YASUSHI ENTERTAINMENT TOKEN (AYET)

Audited by TrueL1 SM Audit

Date: 1 July 2026

✓ AI-Powered Smart Contract Security Review